Milestone: 20GB Reached

We've reached 20GB of stored data — thank you for helping us grow!

Patreon Ko-fi Liberapay
Close
Freedium

- ko-fi
- librepay
- patreon

< Go to the original

Skill	Description
persona-exec-assistant	Manage an executive's schedule, inbox, and communications.
persona-project-manager	Coordinate projects — track tasks, schedule meetings, and share docs.
persona-hr-coordinator	Handle HR workflows — onboarding, announcements, and employee comms.
persona-sales-ops	Manage sales workflows — track deals, schedule calls, client comms.
persona-it-admin	Administer IT — monitor security and configure Workspace.
persona-content-creator	Create, organize, and distribute content across Workspace.
persona-customer-support	Manage customer support — track tickets, respond, escalate issues.
persona-event-coordinator	Plan and manage events — scheduling, invitations, and logistics.
persona-team-lead	Lead a team — run standups, coordinate tasks, and communicate.
persona-researcher	Organize research — manage references, notes, and collaboration.

Google Workspace CLI: Agent Native Alternative to Zapier, n8n and Make

The hardest part about interacting with enterprise tools has always been the plumbing, OAuth dance, pagination.

Agent Native Follow
~10 min read · March 8, 2026 (Updated: March 8, 2026) · Free: No

The twelve different wrapper libraries, each with their own opinions about error handling and retry logic, I have definitely lost entire weekends writing custom Google API integrations that do nothing more interesting than list files in a Drive folder or send an email!

This is the exact pain point Zapier, Make, and n8n built entire businesses around: hiding the ugly integration plumbing behind a nicer interface.

But if you're a developer, that convenience can still feel like another layer standing between you and the API.

So instead of adding another abstraction layer on top, Google recently released `gws`, which reads Google's live API catalog at runtime and builds every command, every flag, every parameter on the fly.

This means that when Google adds a new Sheets endpoint, your CLI will support without updates or waiting for a maintainer to catch up.

It also ships with `Core Google Workspace API skills`, `Helpers`, `Personas`, and `Recipes` that make tools like Zapier and n8n look painfully manual.

Skill	Description
persona-exec-assistant	Manage an executive's schedule, inbox, and communications.
persona-project-manager	Coordinate projects — track tasks, schedule meetings, and share docs.
persona-hr-coordinator	Handle HR workflows — onboarding, announcements, and employee comms.
persona-sales-ops	Manage sales workflows — track deals, schedule calls, client comms.
persona-it-admin	Administer IT — monitor security and configure Workspace.
persona-content-creator	Create, organize, and distribute content across Workspace.
persona-customer-support	Manage customer support — track tickets, respond, escalate issues.
persona-event-coordinator	Plan and manage events — scheduling, invitations, and logistics.
persona-team-lead	Lead a team — run standups, coordinate tasks, and communicate.
persona-researcher	Organize research — manage references, notes, and collaboration.

That is why we are talking about it.

Let's have a look at what this means for anyone building AI agents on top of Google Workspace, and where I think the real tradeoffs hide, because they absolutely do.

> Thanks for reading this article. I'm writing a deep-dive ebook on **Agentic SaaS,** the emerging design patterns that are quietly powering the most innovative startups of 2026.
>
> **Read it here:** [Agentic SaaS Patterns Winning in 2026](), packed with real-world examples, architectures, and workflows you won't find anywhere else.

What Exactly Is `gws`

At its core, `gws` is a single binary (written in Rust, distributed via npm, Cargo, or pre-built binaries) that wraps the entirety of Google Workspace APIs behind composable terminal commands.

```
npm install -g @googleworkspace/cli
```

Copy

That one install gives you programmatic access to Gmail, Drive, Calendar, Sheets, Docs, Chat, Admin, Meet, Forms, Keep, Classroom, Tasks, essentially every Google Workspace service that has a public API.

What makes `gws` fundamentally different from every Google API wrapper is the dynamic command generation architecture.

The Two-Phase Parsing Strategy

When you run a command like:

```
gws drive files list --params '{"pageSize": 10}'
```

Copy

The CLI uses a two-phase parsing strategy:

1. **Phase 1: Service Identification**: Read `argv[1]` (e.g., `drive`) to identify the target service
2. **Phase 2: Dynamic Command Building**: Fetch that service's Discovery Document from Google (cached for 24 hours), then build a `clap::Command` tree from the document's resources and methods
3. Re-parse the remaining arguments against this dynamically generated command tree
4. Authenticate, build the HTTP request, execute

Discovery Service is Google's own machine-readable catalog of every API method, parameter, schema, and OAuth scope across all their services.

It is the same metadata source that Google uses to generate their own client libraries. `gws` just reads it directly and translates it into CLI commands at runtime.

This means the tool literally cannot go out of date as there is no maintenance burden of tracking API changes.

What the Output Looks Like

Every response, e.g. success, error, download metadata, comes back as structured JSON and it is the primary output format, designed to be piped directly into `jq`, parsed by scripts, or consumed by an LLM.

```
# Stream paginated results as NDJSON
gws drive files list --params '{"pageSize": 100}' --page-all | jq -r '.files[].name'

# Preview a request before executing (dry-run)
gws chat spaces messages create \
--params '{"parent": "spaces/xyz"}' \
--json '{"text": "Deploy complete."}' \
--dry-run

# Introspect any method's request/response schema
gws schema drive.files.list
```

Copy

The `--dry-run` flag is something I wish I had known about sooner in every API tool I have ever used. Being able to preview the exact HTTP request before it fires is exactly the kind of safety net you want when an agent is autonomously sending emails or creating calendar events.

How This Changes Agent Development

If you have been building an AI agent that needs to interact with Google Workspace, the status quo genuinely required:

- Setting up OAuth2 credentials with the right scopes
- Writing wrapper functions for each API endpoint you need
- Handling pagination manually (Google's pagination model varies across services)
- Parsing different response shapes from different APIs
- Managing token refresh, credential storage, error handling
- Testing all of this against live APIs because Google's Workspace APIs do not have great sandbox environments

For every new service you want your agent to access, you are looking at hours of boilerplate before your agent can even make its first call.

But with `gws`, you can run one command per action

```
# Send an email
gws gmail +send --to alice@company.com --subject 'Q2 Report Ready' \
--body 'Hey Alice, the Q2 report is ready for review.'

# Upload a file to Drive
gws drive files create --json '{"name": "report.pdf"}' --upload ./report.pdf

# Create a spreadsheet
gws sheets spreadsheets create --json '{"properties": {"title": "Q1 Budget"}}'

# Search unread emails from a specific person
gws gmail users.messages list \
--params '{"userId": "me", "q": "from:boss@company.com is:unread"}'
```

You'd be much more willing to experiment with agent-to-Workspace integrations when the cost of trying something is one terminal command instead of a 200-line Python script.

The MCP Server

> The built-in MCP server mode was removed after this article was written. Since that decision may still be revisited, we've kept this section intact for context. If you need an alternative in the meantime, you can consider Google Workspace MCP

Running:

```
gws mcp -s drive,gmail,calendar
```

starts an MCP server over `stdio` that exposes those Workspace APIs as structured tools.

Plug it into Claude Desktop, VS Code, or any MCP-compatible client, and the LLM can now manage your Workspace natively.

For my own work, the pattern I find most useful is:

1. Authenticate once on my local machine: `gws auth setup`
2. Start the MCP server with the services I need
3. Let the LLM handle email triage, calendar scheduling, or document management through structured tool calls

Running `gemini extensions install https://github.com/googleworkspace/cli` also gives your local Gemini agent direct access to all `gws` commands, inheriting your existing credentials.

100+ Agent Skills: What That Actually Means

The repo ships with over 100 Agent Skills, these are `SKILL.md` files that serve as structured instructions for AI agents.

Core Google Workspace API skills.

Skill	Description
gws-shared	gws CLI: Shared patterns for authentication, global flags, and output formatting.
gws-drive	Google Drive: Manage files, folders, and shared drives.
gws-sheets	Google Sheets: Read and write spreadsheets.
gws-gmail	Gmail: Send, read, and manage email.
gws-calendar	Google Calendar: Manage calendars and events.
gws-admin-reports	Google Workspace Admin SDK: Audit logs and usage reports.
gws-docs	Read and write Google Docs.
gws-slides	Google Slides: Read and write presentations.
gws-tasks	Google Tasks: Manage task lists and tasks.
gws-people	Google People: Manage contacts and profiles.
gws-chat	Google Chat: Manage Chat spaces and messages.
gws-classroom	Google Classroom: Manage classes, rosters, and coursework.
gws-forms	Read and write Google Forms.
gws-keep	Manage Google Keep notes.
gws-meet	Manage Google Meet conferences.
gws-events	Subscribe to Google Workspace events.
gws-modelarmor	Google Model Armor: Filter user-generated content for safety.
gws-workflow	Google Workflow: Cross-service productivity workflows.

They cover every supported API, plus higher-level helpers for common workflows and 50 curated recipes for Gmail, Drive, Docs, Calendar, and Sheets.

You install them with:

```
# Install all skills at once
npx skills add https://github.com/googleworkspace/cli

# Or pick only what you need
npx skills add https://github.com/googleworkspace/cli/tree/main/skills/gws-drive
npx skills add https://github.com/googleworkspace/cli/tree/main/skills/gws-gmail
```

For OpenClaw users, you can symlink them directly:

```
ln -s $(pwd)/skills/gws-* ~/.openclaw/skills/
```

The `gws-shared` skill includes an `install` block so OpenClaw auto-installs the CLI via `npm` if `gws` is not on PATH. (That attention to the install-time experience is something I genuinely appreciate).

Model Armor: The Security Story Nobody Is Talking About

Everyone focuses on the convenience and the star count, but almost nobody is discussing the [Model Armor integration](#).

When your AI agent reads an email via `gws`, the email content becomes part of the agent's context, which is a prompt injection surface.

Someone could embed malicious instructions in an email body or a shared document, and the agent might follow them.

`gws` addresses this with a `--sanitize` flag that routes API responses through [Google Cloud Model Armor](#) before they reach your agent:

```
gws gmail users messages get --params '...' \
--sanitize "projects/P/locations/L/templates/T"
```

Model Armor screens for prompt injection, jailbreaking attempts, sensitive data leaks, and malicious URLs.

You can set it to `warn` mode (log and continue) or `block` mode (reject the response entirely).

Mode	Behavior	Use Case
warn (default)	Log detection, pass response through	Development, monitoring
block	Reject response on detection	Production agent deployments

You configure the default template via environment variable:

```
export GOOGLE_WORKSPACE_CLI_SANITIZE_TEMPLATE="projects/P/locations/L/templates/T"
export GOOGLE_WORKSPACE_CLI_SANITIZE_MODE="block"
```

Model Armor requires its own Google Cloud setup and is not free at scale,.

Model Armor pricing	Pay-as-you-go and subscription options.	
Package	Tier	Pricing
Model Armor standalone	Free tier	Free for up to 2 million tokens/month
	Additional usage over 2 million tokens	$0.10 per additional 1 million tokens
SCC Premium (Organization-level) - Pay-as-you-go	Free tier	Free for up to 2 million tokens/month
	Additional usage over 2 million tokens	$0.10 per additional 1 million tokens
SCC Premium (Project-level) - Pay-as-you-go	Free tier	Free for up to 2 million tokens/month
	Additional usage over 2 million tokens	$0.10 per additional 1 million tokens
SCC Premium - subscription only	Included with subscription	3 billion tokens/month
	Additional usage over 3 billion tokens	$0.10 per additional 1 million tokens
SCC Enterprise - subscription only	Included with subscription	3 billion tokens/month
	Additional usage over 3 billion tokens	$0.10 per additional 1 million tokens

But the fact that response sanitization is a first-class feature of the CLI, not a third-party add-on, shows where Google thinks the security boundaries need to be for agentic workflows.

Authentication: The Part That Can Still Burn You

The authentication story is where `gws` inherits all the traditional complexity of Google's OAuth ecosystem.

The Happy Path is if you have `gcloud` installed, `gws auth setup` walks you through creating a Cloud project, enabling APIs, and logging in. Credentials are encrypted at rest (AES-256-GCM) with the key stored in your OS keyring. This works well for personal use.

The Enterprise Pain is that if you are at a company with a managed Google Workspace, you need a GCP project with OAuth credentials, and it's really not necessary to create a GCP project just to obtain authentication tokens.

Unverified OAuth apps (testing mode) are limited to roughly 25 scopes. The recommended scope preset includes 85+ scopes and will fail for unverified apps.

You need to select individual services:

```
gws auth login -s drive,gmail,sheets
```

Headless / CI is where you complete interactive auth on a machine with a browser, then export credentials:

```
gws auth export --unmasked > credentials.json

# On the headless machine:
export GOOGLE_WORKSPACE_CLI_CREDENTIALS_FILE=/path/to/credentials.json
```

Auth Precedence

# Priority	Source	Set via
1	Access token	GOOGLE_WORKSPACE_CLI_TOKEN
2	Credentials file	GOOGLE_WORKSPACE_CLI_CREDENTIALS_FILE
3	Encrypted credentials	gws auth login
4	Plaintext credentials	~/.config/gws/credentials.json

If you have multiple auth methods configured and something is not working, check this precedence table first.

What This Means for the "Workflow Automation" Market

There is a category of SaaS tools, e.g. Zapier, Make, n8n, Bardeen, and many others, that charge $20-$99/month primarily to bridge "user wants to do X in Gmail/Drive/Calendar" and "here is the API call that does X."

`gws` does not kill those tools but for developers and AI agents, the value proposition of paying a monthly subscription for what is now a free `npm install` gets harder to justify.

The comparison I find more interesting is `gws` versus `gcloud`:

Feature	gws	gcloud CLI
Scope	Google Workspace APIs	All Google Cloud services
Agent Skills	100+ pre-built	None
MCP Support	Native (built-in server)	Manual setup
Output Format	Structured JSON (always)	Variable
Command Generation	Dynamic (Discovery Service)	Static
Workspace-Specific	Yes (purpose-built)	Partial

`gws` is not a replacement for `gcloud`. It is a Workspace-specific tool that happens to be extremely well-suited for AI agent integrations in a way that `gcloud` never tried to be.

Practical Use Cases

Here are some practical use cases to start with.

Agent Email Triage

```
# Get the 5 most recent unread emails, summarized for agent consumption
gws gmail +triage --max 5 --query 'is:unread'
```

The `+triage` helper is a higher-level skill that returns a structured summary rather than raw message data. This is very helpful especially when I am working on agents that need to decide whether to surface an email to a human or handle it autonomously.

Document Management Pipelines

```
# Upload a generated report
gws drive files create --json '{"name": "weekly-report.pdf"}' --upload ./report.pdf

# Share it with the team
gws drive permissions create \
--params '{"fileId": "FILE_ID"}' \
--json '{"role": "reader", "type": "user", "emailAddress": "team@company.com"}'
```

Spreadsheet Data Operations

```
# Read a range
gws sheets spreadsheets values get \
--params '{"spreadsheetId": "ID", "range": "Sheet1!A1:C10"}'

# Append rows
gws sheets spreadsheets values append \
--params '{"spreadsheetId": "ID", "range": "Sheet1!A1", "valueInputOption": "USER_ENTERED"}' \
--json '{"values": [["Name", "Score"], ["Alice", 95]]}'
```

Quick note: Sheets ranges use `!` which bash interprets as history expansion. Always wrap values in single quotes.

Failure Modes

- **"Access blocked" or 403 during login**: Your OAuth app is in testing mode and your account is not listed as a test user. Fix: add yourself in the OAuth consent screen under Test Users
- **"Google hasn't verified this app"**: Expected in testing mode. Click Advanced, then proceed. Safe for personal use
- **Too many scopes error**: Unverified apps cap at around 25 scopes. Use `-s drive,gmail,calendar` instead of the recommended preset
- `**redirect_uri_mismatch`: The OAuth client was not created as a Desktop app type. Delete it, recreate as Desktop, download new JSON
- **API not enabled**: You will see a 403 with `accessNotConfigured`. The CLI helpfully prints the enable URL, click it, enable the API, wait 10 seconds, retry
- **Discovery Document cache**: Cached for 24 hours. If you need to force a refresh after a new API launch, I suspect clearing `~/.config/gws/` cache would work, but this needs more testing

Concluding Thoughts

By reading the Discovery Service at runtime instead of shipping static command definitions, Google solved the API wrapper maintenance problem in a way that no third-party library can match.

They have the advantage of being both the API provider and the CLI builder.

But here's what still needs work:

- Authentication setup is steep for enterprise environments, a GCP project requirement is a real barrier
- "Not officially supported" is a meaningful risk for production deployments
- Pre-v1.0 with breaking changes expected, not ready for stability-critical use cases
- Model Armor requires separate Google Cloud setup and has its own cost implications

I would love to hear from anyone who has deployed this in a team setting or hit walls I have not encountered yet.

Bonus Articles

7 Local LLM Families To Replace Claude/Codex (for everyday tasks)

Open-source model families you can run locally that are now delivering real-world performance surprisingly close to...

medium.com

Zero-Human Company with OpenClaw, Claude, and Codex: How to Build an Agent Organization

The zero-human company orchestrates multiple AI agents (Claude, Codex, OpenClaw) into a structured organization.

medium.com

Qwen 3.5 35B-A3B: Why Your $800 GPU Just Became a Frontier Class AI Workstation

I have been running local models for a while now, and I thought I had a pretty good sense of where the ceiling was for...

medium.com

GET SH*T DONE: Meta-prompting and Spec-driven Development for Claude Code and Codex

GSD ("Get Shit Done") aims to solve context rot, the quality degradation as the model's context window fills.

medium.com

OpenClaw Memory Systems That Don't Forget: QMD, Mem0, Cognee, Obsidian

If your agent has ever randomly ignored a decision you know you told it... it's not random.

medium.com

Codex 5.3 vs. Opus 4.6: One-shot Examples and Comparison

Codex 5.3 vs. Opus 4.6: One-shot Examples and Comparison Just after 9:45 a.m. Pacific on 5 February 2026, Anthropic...

medium.com
#ai-agent#agentic-ai#google#google-cloud-platform#workflow-automation

Reporting a Problem

Sometimes we have problems displaying some Medium posts.

If you have a problem that some images aren't loading - try using VPN. Probably you have problem with access to Medium CDN (or fucking Cloudflare's bot detection algorithms are blocking you).

Problem Description

Describe your problem here...

Submit Cancel